

HW 06 - Money in US politics

Max Asmar

Every election cycle brings its own brand of excitement – and lots of money. Political donations are of particular interest to political scientists and other researchers studying politics and voting patterns. They are also of interest to citizens who want to stay informed of how much money their candidates raise and where that money comes from.

In the United States, *“only American citizens (and immigrants with green cards) can contribute to federal politics, but the American divisions of foreign companies can form political action committees (PACs) and collect contributions from their American employees.”*¹

In this assignment we will scrape and work with data foreign connected PACs that donate to US political campaigns. First, we will get data foreign connected PAC contributions in the 2024 election cycle. Then, you will use a similar approach to get data such contributions from previous years so that we can examine trends over time.

In order to complete this assignment you will need a Chrome browser with the Selector Gadget extension installed.

Getting started

Open the `hw-06-money-in-politics.Rmd` file in your RStudio Notebook on JupyterHub. Knit the document to make sure it compiles without errors.

Note: You will also be working in the `scrape-pac.R` file located in the `scripts` folder. This file contains starter code with blanks for you to fill in.

Learning Objectives

In this assignment, you will apply skills from lecture and lab to:

- Write a custom function to scrape data from a website
- Use iteration to apply your function across multiple webpages
- Clean and reshape scraped data for analysis
- Create visualizations to explore patterns over time

Warm up

Before we introduce the data, let's warm up with some simple exercises. Update the YAML of your R Markdown file with your information, knit, commit, and push your changes. Make sure to commit with a meaningful commit message. Then, go to your repo on GitHub and



Figure 1: Photo by Sharon McCutcheon on Unsplash

¹ Source: Open Secrets - Foreign Connected PACs.

confirm that your changes are visible in your Rmd **and** md files. If anything is missing, commit and push again.

Packages

We'll use the **tidyverse** package for much of the data wrangling and visualisation, the **robotstxt** package to check if we're allowed to scrape the data, the **rvest** package for data scraping, and the **scales** package for better formatting of labels on visualisations. These packages are already installed for you. You can load them by running the following in your Console:

```
library(tidyverse)
library(robotstxt)
library(rvest)
library(scales)
```

Data

This assignment does not come with any prepared datasets. Instead you'll be scraping the data!

Helpful Functions for This Assignment

You've learned these functions in lecture and lab. Here's a quick reference:

Web Scraping Functions: - `read_html(url)` - Read HTML from a webpage - `html_node(css)` - Extract a specific element using CSS selector - `html_table()` - Parse an HTML table into a data frame

String Manipulation Functions: - `str_squish(string)` - Remove extra whitespace - `str_sub(string, start, end)` - Extract part of a string (use negative numbers to count from end) - `str_remove(string, pattern)` - Remove first occurrence of a pattern - `str_remove_all(string, pattern)` - Remove all occurrences of a pattern

Data Reshaping Functions: - `separate(col, into, sep)` - Split one column into multiple columns - `pivot_longer()` - Reshape data from wide to long format

Iteration: - `map_df(list, function)` - Apply a function to each element and combine results into data frame

Remember: Special characters like `$` and `\` need to be escaped with `\\` in R!

Exercises

Data collection via web scraping

The data come from OpenSecrets.org, a “website tracking the influence of money on U.S. politics, and how that money affects policy and citizens’ lives”. This website is hosted by The Center for Responsive Politics, which is a nonpartisan, independent nonprofit that “tracks money in U.S. politics and its effect on elections and public policy.”²

Before getting started, let’s check that a bot has permissions to access pages on this domain.

```
library(robotstxt)
paths_allowed("https://www.opensecrets.org")
```

```
## [1] TRUE
```

Our goal is to scrape data for contributions in all election years Open Secrets has data for. Since that means repeating a task many times, let’s first write a function that works on the first page. Confirm it works on a few others. Then iterate it over pages for all years.

Working with the Starter Code

The `scrape-pac.R` file in your `scripts` folder contains starter code with blanks (___) for you to fill in.

How to approach this: 1. Uncomment the code as you work through each section (remove the # at the beginning of lines) 2. Fill in the blanks based on what you learned in lecture and lab 3. Test your function on individual years before mapping over all URLs 4. Check that your output matches the expected format (see below)

Tips:

- Work section by section
- don’t uncomment everything at once
- Run your code frequently to catch errors early
- Use the test section to verify your function works before mapping

Complete the following set of steps in the `scrape-pac.R` file in the `scripts` folder of your repository. This file already contains some starter code to help you out.

- Write a function called `scrape_pac()` that scrapes information from the Open Secrets webpage for foreign-connected PAC contributions in a given year. This function should



² Source: Open Secrets - About.

- have one input: the URL of the webpage and should return a data frame.
 - rename variables scraped, using `snake_case` naming.
 - clean up the `Country of Origin/Parent Company` variable with `str_squish()`.
 - add a new column to the data frame for `year`. We will want this information when we ultimately have data from all years, so this is a good time to keep track of it. Our function doesn't take a year argument, but the year is embedded in the URL, so we can extract it out of there, and add it as a new column. Use the `str_sub()` function to extract the last 4 characters from the URL. You will probably want to look at the help for this function to figure out how to specify "last 4 characters".
- Define the URLs for 2024, 2020, and 2000 contributions. Then, review the function and URLs below to understand how the data was collected. Does the function seem to do what you expected it to do?

Expected output check: After running `scrape_pac(url_2024)`, your data frame should have:

- 5 columns: `name`, `country_parent`, `total`, `dems`, `repubs`, `year`
- Multiple rows (the number may vary as data updates)
- The `year` column should contain 2022 for all rows
- No extra whitespace in the `country_parent` column

If your output doesn't match this, revisit your function before proceeding.

- Construct a vector called `urls` that contains the URLs for each webpage that contains information on foreign-connected PAC contributions for a given year.
- Map the `scrape_pac()` function over `urls` in a way that will result in a data frame called `pac_all`.
- Write the data frame to a csv file called `pac-all.csv` in the `data` folder.

Checkpoint: Before moving on, verify your data collection worked:

- Check that `pac-all.csv` exists in your `data` folder
- The file should contain data from multiple years (2000-2022)
- Open the CSV file to quickly verify it has the expected columns and data
- You should have data from 12 election cycles (2000, 2002, 2004, ..., 2020, 2022)

If something looks wrong, revisit your scraping code before continuing.

Now is a good time to commit and push your changes to GitHub with an appropriate commit message (e.g. “Completed data cleaning”). Make sure to commit and push all changed files so that your Git pane is cleared up afterwards.

Complete the following set of steps in the `hw-06.Rmd` file in your repository.

```
# This chunk loads the pre-collected dataset.
# You do not need to modify this chunk.
# pac_all is already loaded from the chunk above
```

1. The dataset `pac-all.csv` has been loaded above. Report its number of observations and variables using inline code.

```
pac_all <- read_csv("data/pac-all.csv")

## Rows: 2404 Columns: 6
## -- Column specification -----
## Delimiter: ","
## chr (5): name, country_parent, total, dems, repubs
## dbl (1): year
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

There are 2404 observations and 6 variables in the `pac-all` dataset.

Data cleaning

In this section we clean the `pac_all` data frame to prepare it for analysis and visualization. We have two goals in data cleaning:

- Separate the `country_parent` into two such that country and parent company appear in different columns for country-level analysis.
- Convert contribution amounts in `total`, `dems`, and `repubs` from character strings to numeric values.

The following exercises walk you through how to make these fixes to the data.

2. Use the `separate()` function to separate `country_parent` into `country` and `parent` columns.
 - Country and parent company names are separated by / (forward slash)

- Use `sep = "/"` in your `separate()` function
- Some entries have multiple / characters (e.g., "Denmark/Novo Nordisk A/S")
- Set `extra = "merge"` so these split into only 2 columns at the first /
- Example: "Denmark/Novo Nordisk A/S" becomes `country = "Denmark"` and `parent = "Novo Nordisk A/S"`

```
pac_all <- pac_all %>%
  separate(
    col = country_parent,
    into = c("country", "parent"),
    sep = "/",
    extra = "merge"
  )
# Print top 10 rows to verify
pac_all

## # A tibble: 2,404 x 7
##   name                country parent total dems repubs year
##   <chr>              <chr>   <chr> <chr> <chr> <dbl>
## 1 7-Eleven          Japan   Ito-Y~ $8,5~ $1,5~ $7,000 2000
## 2 ABB Group         Switze~ Asea ~ $46,~ $17,~ $28,5~ 2000
## 3 Accenture         UK      Accen~ $75,~ $23,~ $52,9~ 2000
## 4 ACE INA           UK      ACE G~ $38,~ $12,~ $26,0~ 2000
## 5 Acuson Corp (Siemens AG) Germany Sieme~ $2,0~ $2,0~ $0      2000
## 6 Adtranz (DaimlerChrysler) Germany Daiml~ $10,~ $10,~ $500    2000
## 7 AE Staley Manufacturing (Tate & Lyle) UK      Tate ~ $24,~ $10,~ $14,0~ 2000
## 8 AEGON USA (AEGON NV)   Nether~ Aegon~ $58,~ $10,~ $47,7~ 2000
## 9 AIM Management Group  UK      AMVES~ $25,~ $10,~ $15,0~ 2000
## 10 Air Liquide America  France  L'Air~ $0     $0     $0      2000
## # i 2,394 more rows
```

Verify: Check that `total`, `dems`, and `repubs` now show as numbers (not text) and have no \$ or , characters.

Write a sentence describing what you observe in the separated columns.

I observe that the `country_parent` column has been reorganized into distinct `country` and `parent` columns. In the `country` column I see the country where the PAC donation originated as a char variable (categorical), and in the `parent` column I see the names of parent companies as a char variable (categorical).

3. Remove the character strings including \$ and , signs in the `total`, `dems`, and `repubs` columns and convert these columns to numeric.

End your code chunk by printing out the top 10 rows of your data frame (if you just type the data frame name it should automatically do this for you). A couple hints to help you out:

- The `$` character is a special character in R and must be escaped as `"\\$"`
- Use `str_remove()` to remove the `$` sign
- Use `str_remove_all()` to remove all commas (since there can be multiple in large numbers)
- After removing characters, use `as.numeric()` to convert to numbers
- Apply this to all three columns: `total`, `dems`, and `repubs`

You can either use `mutate()` three times (once for each column) or write the same transformation three times within one `mutate()` call.

```
pac_all <- pac_all %>%
  mutate(
    total = total %>% str_remove("\\$") %>% str_remove_all(",") %>% as.numeric(),
    dems   = dems   %>% str_remove("\\$") %>% str_remove_all(",") %>% as.numeric(),
    repubs = repubs %>% str_remove("\\$") %>% str_remove_all(",") %>% as.numeric()
  )
```

Print top 10 rows to verify

```
pac_all

## # A tibble: 2,404 x 7
##   name                country parent total  dems repubs  year
##   <chr>              <chr>   <chr> <dbl> <dbl> <dbl> <dbl>
## 1 7-Eleven          Japan   Ito-Y~ 8500  1500   7000  2000
## 2 ABB Group         Switze~ Asea ~ 46000 17000  28500  2000
## 3 Accenture         UK      Accen~ 75984 23000  52984  2000
## 4 ACE INA           UK      ACE G~ 38500 12500  26000  2000
## 5 Acuson Corp (Siemens AG) Germany Sieme~ 2000  2000    0  2000
## 6 Adtranz (DaimlerChrysler) Germany Daiml~ 10500 10000   500  2000
## 7 AE Staley Manufacturing (Tate & Lyle) UK      Tate ~ 24000 10000  14000  2000
## 8 AEGON USA (AEGON NV)  Nether~ Aegon~ 58250 10500  47750  2000
## 9 AIM Management Group  UK      AMVES~ 25000 10000  15000  2000
## 10 Air Liquide America  France  L'Air~    0    0    0  2000
## # i 2,394 more rows
```

Now is a good time to knit your document, commit, and push your changes to GitHub with an appropriate commit message. Make sure to commit and push all changed files so that your Git pane is cleared up afterwards.

Data visualization and interpretation

4. Create a line plot of total contributions from all foreign-connected PACs in the Canada and Mexico over the years. Once you have made the plot, write a brief interpretation of what the graph reveals. Few hints to help you out:

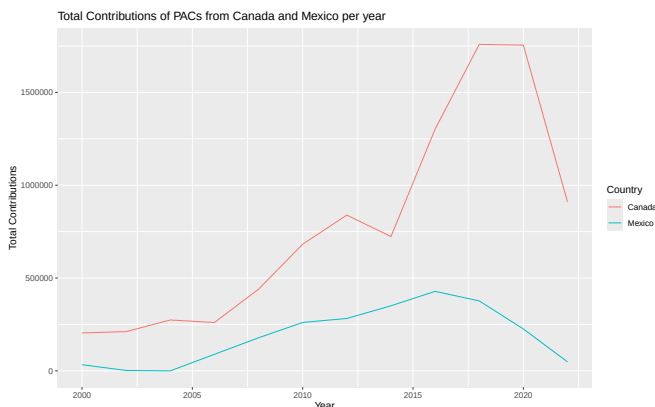
Requirements: - Filter for only Canada and Mexico

- Calculate sum of total contributions by country and year
- Plot: year (x-axis), total contributions (y-axis), separate lines by country (use color)

Note: The figure you create might look slightly different than this one if the data on the website has been updated recently.

```
pac_all %>%
  filter(country %in% c("Canada", "Mexico")) %>%
  group_by(country, year) %>%
  summarize(total_contribution = sum(total)) %>%
  ggplot(aes(x = year, y = total_contribution, color = country)) +
  geom_line() +
  labs(
    title = "Total Contributions of PACs from Canada and Mexico per year",
    x = "Year",
    y = "Total Contributions",
    color = "Country"
  )

## `summarise()` has grouped output by 'country'. You can override using the
## `.groups` argument.
```



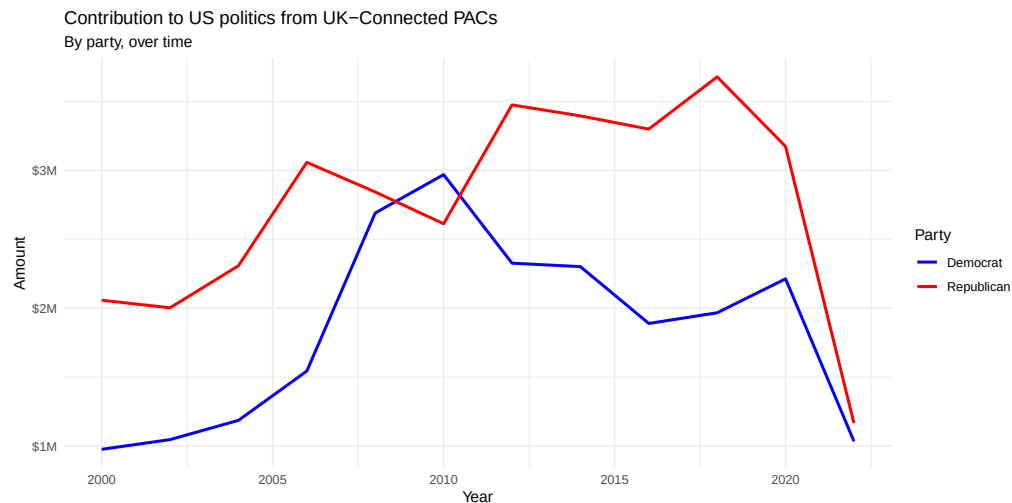
Interpretation (2-3 sentences): Describe the trends you observe. Which country contributes more? How have contributions changed over time?

I observe that Canada consistently contributes more to American PACs than Mexico does. I also notice that generally whenever Canada

increases its PAC contributions, Mexico does the same, and when Canada decreases its PAC contributions, Mexico also does. I notice that there was the greatest net increase in PAC contributions between 2007 and 2017 and the greatest net decrease from around 2016 to the present.

5. The following code creates a visualization showing UK contributions to Democratic and Republican parties over time.

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



- a. What does the `pivot_longer()` function do in this code and why is it necessary for this visualization?

In this code chunk, the `pivot_longer()` function merges the Democrat and Republican variables into 1 single categorical variable called “party”. This is important for the visualization because the ggplot2 library requires that color be a single variable, and it makes the dataset more tidy.

- b. What would happen if you removed the `group = party` argument from `geom_line()`?

If you removed the `group = party` argument from the `geom_line()` function, ggplot2 would not connect the points lining up with their respective party with a trendline.

- c. Modify the code to show contributions from Canada instead of the UK. What do you observe? Copy-paste the code below and make your changes.

```
pac_all <- read_csv("data/pac-all.csv")

parse_currency <- function(x){
  x %>%
    # remove dollar sign
    str_remove("\\$") %>%
    # remove all occurrences of commas
    str_remove_all(",") %>%
    # convert to numeric
    as.numeric()
}

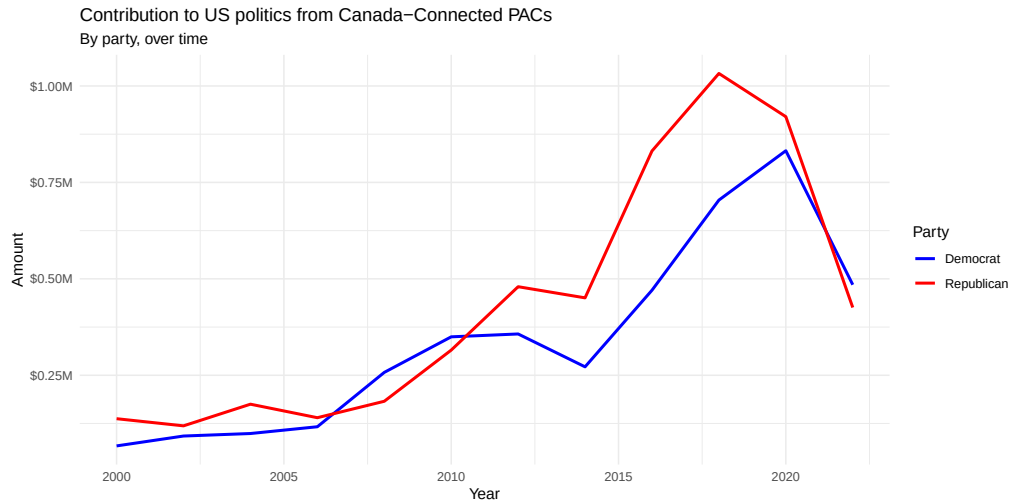
# plot
pac_all %>%
  separate(country_parent, into = c("country", "parent"), sep = "/", extra = "merge") %>%
  mutate(
    total = parse_currency(total),
    dems = parse_currency(dems),
    repubs = parse_currency(repubs)
  ) %>%
  filter(country == "Canada") %>%
  group_by(year) %>%
  summarise(
    Democrat = sum(dems),
    Republican = sum(repubs),
    .groups = "drop"
  ) %>%
  pivot_longer(
    cols = c(Democrat, Republican),
    names_to = "party",
    values_to = "amount"
  ) %>%
  ggplot(aes(x = year)) +
  geom_line(aes(y = amount, group = party, color = party), size = 1) +
  scale_color_manual(values = c("blue", "red")) +
  scale_y_continuous(labels = dollar_format(scale = 0.000001, suffix = "M")) +
  labs(
    x = "Year",
    y = "Amount",
    color = "Party",

```

```

title = "Contribution to US politics from Canada-Connected PACs",
subtitle = "By party, over time"
) +
theme_minimal()

```



Interpretation (2-3 sentences): What patterns do you observe in UK contributions to each party over time?

UK companies tend to contribute more to the American Republican party instead of the Democratic party. However, the only time between the years 2000 and 2020 where this was not the case was between approximately 2008 and 2011, where more money was donated to the Democratic Party. After 2020, however, the UK contributions to both the Democratic and Republican parties have dropped.

Knit, *commit*, and *push* your changes to GitHub with an appropriate commit message. Make sure to commit and push all changed files so that your Git pane is cleared up afterwards and review the md document on GitHub to make sure you're happy with the final state of your work.

Now go back through your write up to make sure you've answered all questions and all of your R chunks are properly labeled.

Once you decide that you are done with the homework, choose the knit drop down and select **Knit to tufte_handout** to generate a pdf. Download and submit that pdf to Canvas.

Common Errors and Troubleshooting

Error: "cannot open the connection" or HTTP 403 when running `scrape_pac()`

- **This is expected.** OpenSecrets.org blocks automated requests

from JupyterHub and most personal machines. You do not need to run the scraping code — use the provided `data/pac-all.csv` instead and start from Exercise 1.

Error: “could not find function ‘read_html’”

- **Solution:** Make sure you’ve loaded the `rvest` package: `library(rvest)`

Error: “Selector gadget not finding the right element”

- **Solution:** The CSS selector is `.DataTable-Partial` - this should be entered as a string in quotes

Error: “No matches for selector”

- **Solution:** Check that you’re using the correct URL format and that the website structure hasn’t changed

Your data frame has weird column names

- **Solution:** Double-check your `rename()` section - column names should match exactly what’s in the scraped table

Getting errors when testing your function

- **Solution:** Make sure you defined your function first (ran the entire `scrape_pac <- function(url) {...} block`)

The year column shows characters instead of just the year

- **Solution:** Use `str_sub(url, -4)` to extract the last 4 characters (the `-4` counts from the end)

`map_df()` returns an error

- **Solution:** Verify your function works on individual URLs first. If it does, check that `urls` is defined correctly